

Advanced Certificate Programme in Agentic AI & RAG Engineering

LLM App Foundations + Model Landscape
Phase 1 · Foundations, Eval Literacy & System Design



Today's outline

Day 1 · Open the body

Concepts

What you'll learn

Tokens, cost, latency 20 min

tiktoken mini-topic 10 min

Three ways to structure output 30 min

Live demos

Watch it built

Tokenizer + cost estimate 10 min

Parse vs JSON-mode vs tool-calling 30 min

Wire tool-calling into ask_llm 25 min

Lab Step 1 starts in class

Build yours

Richer Answer model

Tool-calling ask_llm

schema_version='v1'

Same /ask contract — richer body

Where we are

You have an API. Today we open its body.

BY END OF W3

A stable surface

/ask, /ask_batched, /health all working
Pydantic validation at the boundary
Streaming endpoint — simulated word-by-word
Answer = {content, cost_usd: 0.0001, retries}
ADR 0002 locks the contract



BY END OF W4

Production-grade insides

Tool-calling → structured Answer
Real LLM streaming (no more sleep)
Real cost_usd from response.usage
Two-model comparison: gpt-4o-mini vs gpt-4o
schema_version='v1' — same /ask contract

Tokens, Cost, Latency

Before you write code, you should be able to estimate.

Tokens, Cost, Latency

What is a token?

A token is roughly a sub-word — the unit your model thinks in.

RULE OF THUMB

1 token $\approx$ 4 characters in English

Other languages tokenise differently — non-Latin scripts often use more tokens per character.

Code, JSON, and structured text tokenise oddly — sometimes way more, sometimes less.

Always confirm against the actual tokenizer for the model you're using.

LIVE DEMO • ON SCREEN

OpenAI tokenizer walkthrough

Type a 50-word paragraph → watch the token count.

Switch language to Hindi or Mandarin → token count jumps.

Paste a Python snippet → see the structured-text tokenisation oddities.

→ See Lab Guide Step 0 for the URL.

Tokens, Cost, Latency

Cost = (input + output tokens) × price-per-million

$$\text{cost} = (\text{prompt_tokens} \times \text{in_rate}) + (\text{completion_tokens} \times \text{out_rate})$$

Order-of-magnitude USD per 1M tokens — confirm current rates before quoting.

Model	Input (\$/M)	Output (\$/M)	Best for
gpt-4o-mini	≈ \$0.15	≈ \$0.60	Default for the lab
gpt-4o	≈ \$2.50	≈ \$10.00	Harder reasoning / long context
Claude Sonnet	≈ \$3.00	≈ \$15.00	Long context, strong drafts
Llama 3 (Ollama)	\$0	\$0	Local — see take-home activity

Tokens, Cost, Latency

Latency: TTFT vs total

Two numbers matter — when the user sees the first token, and when the answer is done.

TTFT

Time to First Token

How long the user waits before anything appears. Streaming makes TTFT the felt latency. Typical: 200–800 ms.

Inter-token

Between tokens

The gap between successive tokens once streaming starts. Steady $\approx 10\text{--}50$ ms per token in production.

Total time

Full response

$\text{TTFT} + (\text{tokens_out} \times \text{inter-token})$. For a 200-token answer, often 3–6 s end-to-end.

Tokens, Cost, Latency

tiktoken — count before you call

response.usage tells you tokens after the call. tiktoken tells you tokens before.

Useful when you need to guardrail context length — and essential when retrieved context lands in W6.

```
# Pre-call: count prompt tokens
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o-mini")
n_in = len(enc.encode(prompt))

# Post-call: actual usage from the API
resp = await client.chat.completions.create(...)
usage = resp.usage # .prompt_tokens, .completion_tokens
```

PRE-CALL — tiktoken

Use to guardrail prompt size before sending. Reject or truncate if too long.

POST-CALL — response.usage

Authoritative — what the API actually billed you for. Use for cost tracking.

IN THE LAB

→ Step 2 uses BOTH. tiktoken for the input guard, response.usage for the real cost_usd.

Tokens, Cost, Latency · SUMMARY

The five things you now know

1

Tokens

A token is roughly 4 characters — but always confirm with the actual tokenizer.

2

Cost formula

$\text{Input} \times \text{in-rate} + \text{output} \times \text{out-rate}$. Both halves cost. Long prompts AND long outputs.

3

Cost variance

gpt-4o vs gpt-4o-mini is ~15× cheaper on input. Pick the cheaper model unless quality fails.

4

TTFT vs total

Users feel TTFT, not total. Streaming wins UX even when total time is the same.

5

tiktoken vs usage

tiktoken before the call (guardrail). response.usage after the call (truth).

Three Ways to Force Structured Outputs

Pydantic+parse · JSON mode · Tool-calling — and the rule for choosing.

Structured Outputs

Why this matters

LLMs talk in free text. Your code wants typed objects.

Every step after generation — validation, storage, evaluation, retrieval, tool use — assumes shape.

WITHOUT STRUCTURE

Brittle. Surprising.

LLM adds a friendly preamble. Your parser explodes.

Field misspelled? Silently dropped, three hops later.

Cost of testing every downstream consumer for every LLM update.

WITH STRUCTURE

Predictable. Verifiable.

The shape is the contract. Validation catches drift at the door.

Downstream code reads typed fields, not parsed strings.

LLM swaps in W4+ don't break consumers.

Structured Outputs

Three approaches — at a glance

1

Pydantic + parse

BRITTLE

Ask in free text. Hope it's JSON. Parse and validate. The fallback for older or open-weights models.

2

JSON mode

GUARANTEED JSON

Provider guarantees the body parses as JSON. You still validate with Pydantic. The default for fixed shapes.

3

Tool-calling

STRUCTURED ARGS

Define a tool/function schema. The LLM returns structured arguments. The cleanest path — what we use in the lab.

Structured Outputs • Approach 1

Pydantic + parse — the free-text path

```
# Ask in plain English; hope for JSON
messages = [
    {"role": "user",
     "content": "Answer in JSON: ..."},
]

resp = await client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
)

# Parse the string — where it breaks
raw = resp.choices[0].message.content
data = json.loads(raw)      # ← can throw
ans = Answer.model_validate(data)
```

WORKS WHEN

The LLM is well-behaved.

Prompt is unambiguous.

JSON-mode and tool-calling aren't available (e.g., some open-weights).

BREAKS WHEN

LLM adds 'Here is your JSON:' preamble.

Trailing commas, smart quotes, comments.

Required field renamed or dropped.

Structured Outputs · Approach 1 · The Failure Mode

What you'll see when the parse breaks

```
# The LLM's reply (verbatim)
Here is your JSON answer:

{
  "content": "RAG combines retrieval with generation...",
  "confidence": 0.9,
  "sources": []
}

Hope this helps!
```

`json.loads(raw) → JSONDecodeError: Expecting value: line 1 column 1 (char 0)`

The parser sees 'Here' before the '{' and gives up. Defensive fix: strip preamble with regex — brittle in a different way. This is why structured-output APIs exist. The next two approaches don't have this failure mode.

Structured Outputs • Approach 2

JSON mode — provider guarantees parseable JSON

```
# Same call — one new parameter
resp = await client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    response_format={"type": "json_object"}, # ← the magic
)

# Now `resp.choices[0].message.content` is guaranteed
# to be valid JSON. You still validate the shape:
data = json.loads(resp.choices[0].message.content)
ans = Answer.model_validate(data)
```

WHAT YOU GAIN

The JSON is parseable — no preamble, no trailing notes.

Pydantic still validates field-by-field.

Works for a single fixed shape.

WHAT YOU DON'T

No schema enforcement — fields may be wrong type or missing.

No native way to express 'one-of-these-shapes' (e.g., answer OR clarifying question).

Awkward for multi-step / multi-action outputs.

Structured Outputs · Approach 3 · What We Use

Tool-calling — declare the shape upfront

```
tools = [{
  "type": "function",
  "function": {
    "name": "answer_question",
    "parameters": {
      "type": "object",
      "properties": {
        "content": {"type": "string"},
        "confidence": {"type": "number"},
        "sources": {"type": "array"},
      },
    },
    "required": ["content", "confidence", "sources"]
  },
}]
```

WHY THIS IS THE CLEANEST

Schema declared upfront — model can't drift.

Required fields are required at the provider level.

Multi-shape easy — define multiple tools, model picks one.

The W4 lab pipes the tool call straight into a Pydantic Answer.

Future-proof for W19+ agents (tools become real actions, not just shapes).

Structured Outputs • Comparison

Three approaches — side by side

	Pydantic + parse	JSON mode	Tool-calling
Setup	One extra prompt instruction	One extra param	Define tools schema
Output guarantee	Free text — hope for JSON	Valid JSON	Schema-conformant args
Field validation	Pydantic afterwards	Pydantic afterwards	Provider + Pydantic
Multi-shape	Hard — prompt gymnastics	Hard — one shape	Trivial — many tools
Use it when	Older / open-weights models	Single fixed shape	Anything richer; default
Breakage risk	Highest	Low	Lowest

Structured Outputs • Decision Rule

Which one — in three sentences

DEFAULT

Reach for tool-calling.

Cleanest schema enforcement. What the W4 capstone lab uses for ask_llm.

ALSO FINE

JSON mode — for a single fixed shape.

Slightly less ceremony than tool-calling; fine when you only need one Answer shape.

FALL BACK TO

Pydantic + parse — only when neither is available.

Older models, some open-weights. Expect to write defensive parsing.

Structured Outputs • Live Demo

Three demos — see each fail or succeed

On screen — ~10 minutes per approach. Same question, three different request shapes.

Demo 1

Pydantic + parse → fails

We prompt for JSON, the model adds 'Here is your JSON:'. `json.loads` raises. The classic failure mode.

Demo 2

JSON mode → succeeds

Same prompt, `response_format={'type':'json_object'}`. The parser succeeds. Pydantic still validates the shape.

Demo 3

Tool-calling → succeeds with schema

Define a tool with content/confidence/sources. The model returns structured arguments — no parse step at all.

→ Full walkthrough lives in the Lab Guide. The reference code is at [cohort-repo/week4/reference/](#).

Lab Step 1

Wire tool-calling into ask_llm — richer Answer, same contract

WHAT YOU'RE BUILDING

Tool-calling ask_llm

- 1 Define the answer_question tool**
Schema with content, confidence, sources
- 2 Switch ask_llm to use tools**
Pass tools=[...] to client.chat.completions.create
- 3 Parse the tool call into Answer**
Build Answer(content=..., confidence=..., sources=[...], schema_version='v1')
- 4 Keep cost_usd + retries**
Additive — old fields stay, new fields land

SUB-STEPS · IN THE LAB GUIDE

Seven-beat shape

- 1a** Read the W3 ask_llm + Answer model
- 1b** Define the answer_question tool schema
- 1c** Update Answer with confidence + sources + schema_version
- 1d** Switch ask_llm to pass tools=[...]
- 1e** Parse the tool_call into Answer
- 1f** Smoke test — curl /ask_batched
- 1g** Verify /docs reflects the new shape

Sandbox

Finish 1f → 1g tonight. Day 2 starts with a richer Answer.

Open time before Day 2. Use it to make sure Lab Step 1 is green for you.

WHAT 'GREEN' LOOKS LIKE

- ✓ `curl -X POST .../ask_batched -d '{"question":"What is RAG?"}'` returns content + confidence + sources + schema_version
- ✓ schema_version is the string 'v1'
- ✓ /docs shows the new Answer shape under POST /ask_batched response
- ✓ Old fields (cost_usd, retries) still in the response — additive change, not breaking
- ✓ W3 tests still pass — pytest is green

Day 1 • Wrap

Three things you learned

Cost is a number you compute

Input × in-rate + output × out-rate.
response.usage tells you the truth after the call; tiktoken before.

Three ways to structure output

Parse, JSON mode, tool-calling. Default to tool-calling. The capstone now does.

API contract held

Answer gained confidence + sources + schema_version. cost_usd and retries stayed. /ask is unchanged.

TOMORROW • DAY 2

Real LLM streaming end-to-end. Real cost from response.usage. Two-model comparison.
Schema versioning rule into the ADR. By Friday: your /ask defends itself at DR #1.

Day 2 · Stream it, switch it, defend it

Today's outline

Concepts

What you'll learn

Real streaming end-to-end 20 min
Production timeouts 15 min
Model landscape + Anthropic 25 min
Schema versioning bridge 15 min

Live demos

Watch it built

Stream from OpenAI 15 min
Anthropic switching sidebar 5 min
Cost from response.usage 15 min

Lab Steps 2–4

Build yours

Step 2: streaming + real cost
Step 3: two-model comparison
Step 4: ADR — schema versioning + budget

Streaming, retries, and timeouts

From simulated word-by-word to actual LLM chunks. End-to-end this time.

Streaming

From simulated to real

WHAT W3 SHIPPED

Simulated streaming

Pipeline returns the full answer in one shot
/ask iterates over `.split('')` and yields word-by-word
`asyncio.sleep(0.05)` between yields
User sees streaming — but the latency budget is still the full call
TTFT = time until the entire LLM call finishes

WHAT W4 SHIPS

Real LLM streaming

Pipeline calls `client.chat.completions.create(stream=True)`
Iterates over async chunks as they arrive
Each chunk's `delta.content` is yielded immediately
No more `asyncio.sleep` — gap is real inter-token latency
TTFT drops dramatically — the user sees tokens within ~500 ms



Streaming

How it actually works — delta chunks

```
# stream=True returns an async iterator
stream = await client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    stream=True,                    # ← the magic
)

# Each chunk is a small delta
async for chunk in stream:
    delta = chunk.choices[0].delta
    if delta.content:
        yield delta.content        # arrives ~10-50 ms apart
```

WHAT YOU GET

Each chunk = one (or a few) tokens.

delta.content holds the text increment.

The first chunk often arrives in 200–800 ms.

Subsequent chunks every ~10–50 ms.

Total time $\approx$ TTFT + (tokens $\times$ inter-token).

The async generator shape from W3 stays — only what fills it changes.

Streaming • End-To-End

Same async generator. New source.



THE PRODUCTION LESSON

The W3 ADR's promise holds: same `/ask` contract, transformed body. Streaming is an additive change — clients that didn't expect chunks still get the full text by reading the whole stream.

Streaming • Timeouts

Two timeouts you need — per-call and total

Per-call timeout

30 seconds

Caps any single LLM call. Catches a hung connection or a model stuck in a loop.

Total batch timeout

5 minutes

Caps the whole batch when you're processing many questions in parallel.

Mid-stream retry

Generally don't

Retrying a streaming call after partial output is hard. Usually: retry the whole call from the start.

Streaming • Live Demo

15 lines, end-to-end

On screen — the streaming flow from OpenAI through ask_llm through FastAPI's StreamingResponse to your terminal.

WHAT YOU'LL SEE

- 1 Open the W4 reference `api/main_reference.py`** `stream_answer` becomes an async generator over real chunks
- 2 Start `uvicorn`** Watch logs for HTTP requests
- 3 `curl --no-buffer`** First token visible in ~500 ms
- 4 Tweak the model parameter** Swap `gpt-4o-mini` for `gpt-4o`; see latency change

→ Detailed walkthrough in the Lab Guide Step 2. The reference code is at `<cohort-repo>/week4/reference/api_main_reference.py`.

Model Landscape

What's out there, what we pick, and how you'd switch.

Model Landscape

Six options worth knowing

Model	Type	Rough strength	Rough cost	Reach for it when
gpt-4o-mini	Hosted (OpenAI)	Good general / fast	Cheapest cloud	Default for the W4 lab
gpt-4o	Hosted (OpenAI)	Strong reasoning	~15× gpt-4o-mini	Hard questions, long context
Claude Sonnet 4	Hosted (Anthropic)	Excellent drafts, long ctx	Comparable to gpt-4o	See switching sidebar
Gemini 1.5 Pro	Hosted (Google)	2M context window	Mid-tier	Very long documents
Llama 3 (8B/70B)	Open-weights	Solid baseline	Hosting cost only	Self-host for privacy
Llama 3 via Ollama	Local (open-weights)	Modest but useful	\$0 marginal	→ See take-home activity

Cost numbers go stale fast. Confirm current rates at <https://artificialanalysis.ai> before quoting.

Model Landscape • Lab Comparison

Two OpenAI models, side by side

DEFAULT • THE WORKHORSE

gpt-4o-mini

≈ \$0.15 / 1M input, \$0.60 / 1M output

Fast — TTFT often under 500 ms

Handles 80% of capstone questions well

Use as the default in your lab pipeline

ESCAPE HATCH • WHEN MINI ISN'T ENOUGH

gpt-4o

≈ \$2.50 / 1M input, \$10.00 / 1M output

Stronger reasoning, better drafts

Use for harder questions or when mini's confidence is low

In the lab: same 10 questions through both — eyeball the gap

Model Landscape · Sidebar · Reference Only

Switching to Anthropic — three differences

We use OpenAI in the lab. Here's what changes if you ever swap to Claude — read once, don't memorise.

1

Client

`anthropic.AsyncAnthropic()` instead of `openai.AsyncOpenAI()`

`client.messages.create(...)` instead of `client.chat.completions.create(...)`

2

Message shape

system prompt is a top-level parameter, not a message role

messages list has only user / assistant — no 'system' inside

3

Streaming + tools

`stream=True` is the same idea, different event shape

Tool-use is via 'tool_use' content blocks, parsed differently

→ A working snippet is at `<cohort-repo>/week4/reference/anthropic_reference.py` — read it; don't run it in the lab.

Model Landscape • Local Models

Ollama — running models on your laptop

Free. Private. Useful for development iteration when you don't want to spend tokens.

Not in the lab — the Vocareum environment doesn't run Ollama. Take it home as an activity.

WHY LOCAL MATTERS

\$0 per call — iterate without spending tokens.

No data leaves your machine — useful for sensitive prompts.

Works offline.

Latency on a recent laptop is usable for development.

Won't match cloud quality — that's the point of comparing.

IN THE TAKE-HOME ACTIVITY

Install Ollama on your laptop (~5 min)

Pull llama3.2:3b (~2 GB disk)

Make your first local call from the W4 pipeline

Run the lab's 10 questions through it

Compare cost + quality against your lab's gpt-4o-mini results

Model Landscape • Live Demo

Cost from response.usage — live

On screen — replacing the W2 placeholder `cost_usd=0.0001` with the real number from the API response.

WHAT YOU'LL SEE

- | | | |
|---|----------------------------------|--|
| 1 | Read response.usage | <code>prompt_tokens, completion_tokens, total_tokens</code> |
| 2 | Multiply by current rates | Build a small <code>RATES</code> dict in <code>cost.py</code> |
| 3 | Surface real cost_usd | <code>Answer.cost_usd</code> is now a real number, not 0.0001 |
| 4 | Extend SQLite | Add <code>cost_usd</code> + <code>model</code> columns to the <code>answers</code> table |

→ Detailed walkthrough in the Lab Guide Step 2. Reference code at `<cohort-repo>/week4/reference/cost.py`.

Schema Versioning

The Answer shape will change in W6, W13, W19. The contract must hold.

Schema Versioning

Why this matters across the rest of the programme

Today's Answer = {content, cost_usd, retries, confidence, sources, schema_version='v1'}.
It won't stay that way.

W4

v1

content, confidence, sources,
schema_version

W6

v2

+ retrieved_chunks, retrieval_score

W13

v3

+ tool_calls_made, action_trace

W19

v4

+ agent_steps, reasoning_chain

Without a versioning rule, every change above is a downstream consumer breakage waiting to happen.

Schema Versioning · The Fix

Three rules — that's all you need

1

Stamp every output with ``schema_version``

A simple string field on the Answer model. "v1" today, "v2" when you ship breaking changes.

2

Add optional. Remove never (without a bump).

New fields with defaults are safe — old consumers ignore them. Removing or renaming is a bump.

3

Support both during transitions

When v2 ships, /ask still returns v1 by default. A header or query param opts into v2. Two weeks parallel min.

Brief note on Avro / Protobuf: same principles, more ceremony. JSON + Pydantic carries us through W30.

Schema Versioning · The ADR Rule

Additive (no bump) vs Breaking (bump)

ADDITIVE · NO BUMP

Safe under v1

- New optional field with a default
- New endpoint at a different path
- Internal model change (W4 is exactly this)
- Retry / logging / observability changes
- Performance improvements
- Prompt edits — same shape out

BREAKING · BUMP TO v2

Forces a v2

- Removing a required field
- Renaming a public field
- Changing a field's type
- Optional → required
- Semantic change (cost_usd suddenly means tokens not USD)
- Changing error-response shape

Schema Versioning • ADR

Write the rule into your ADR — one paragraph

ADR addition — Schema versioning rule

Every Answer carries a `schema_version` field.

Today's shape is v1. Additive changes (new optional fields, new endpoints, internal model swaps) ship without a bump and stay under v1.

Breaking changes (removed fields, renamed fields, type changes) ship as a new `schema_version` (v2, v3, ...) and run in parallel with the previous version for at least two weeks.

Consumers can request a specific version via a header or query parameter; default is the oldest still-supported version.

Cost budget: capstone answers $\leq$ \$0.01 each on average (will refine after Lab Step 3).

Lab Steps 2–4

Real streaming, real cost, real comparison, real ADR

Step 2

60 min

Streaming + real cost

Swap simulated streaming for OpenAI chunks. `response.usage` → `cost_usd`. Extend SQLite answers table with `cost_usd` + `model` columns.

Step 3

45 min

Two-model comparison

Run the same 10 questions through gpt-4o-mini AND gpt-4o. Save both result sets. Eyeball the gap. Write a 2-paragraph note in `docs/lab4-model-comparison.md`.

Step 4

15 min

Update the ADR

Schema versioning rule. Chosen model + reason. Cost budget ($\leq \$0.01/\text{answer}$). Note that the W3 contract held.

Week 4 • Wrap

Your /ask now defends itself.

Five things you can now do — and what's coming.

1

Estimate cost before writing code

2

Force structured output three ways

3

Stream end-to-end, real chunks

4

Compare two models on the same Qs

5

Version the schema for W6+

NEXT WEEK • W5

Milestone 1 — Design Review

Prompting + Eval Literacy + Stakeholder framing. By end of W5: your capstone has been defended once.

Q & A

Thank you